

DCP Provider Fleet Operations.

Who owns install, machine introspection, self-heal, verification, and fleet-wide monitoring across every provider machine — and how we manage, maintain, and prove the whole system is running on each one.

DESIGN DOC · V1

Operating principle — claimed vs. earned. A provider box is never trusted to declare its own health. Self-reported signals (heartbeats, status flags) are advisory; truth is **earned** by an independent probe — a real inference, a kernel-level tunnel check — from *outside* the box. Every "online" claim derives from earned state, and every capacity/liveness gap fails **loud**, not silent.

1 Ownership map — who handles what

JOB	OWNER	HOW
Install / bootstrap	Installer script (desktop = GUI)	One command/click: detect OS+GPU → install engine (Ollama/vLLM/llama.cpp/MLX) → download the daemon (single source) → set up WireGuard → register (single-use token→key) → install as a system service that survives reboot. <i>Not the AI.</i>
"What GPU? What's running? Is the engine up? Everything installed?"	The daemon, continuously	Multi-layer GPU detection (model/VRAM/driver); engine discovery + watchdog (knows + restarts Ollama :11434 / vLLM :8000 / llama.cpp :8080 / MLX); cached-model list; disk; a local health endpoint :19876 .
Self-heal (restart engine, fix tunnel)	The daemon	Engine watchdog + WireGuard self-heal + crash-rollback on a bad self-update.
"Is this box actually serving?"	Backend verification providerVerification.js	Independent probe: real GET /v1/models + 1-token inference + WG handshake age (kernel truth). Earned, not claimed.
"Is the whole fleet healthy? Did anyone go dark?"	Off-box watchdog + /admin/fleet + alerting	Dead-man's-switch (survives the box <i>and</i> the backend dying); per-provider state machine that fires an event on online→offline and notifies the provider + the platform.
Keep every machine current	Daemon self-update + heartbeat tasks	Push one daemon version → fleet self-updates (sha256-verified + rollback) on next heartbeat. Config/commands (run_mode, pull, drain) ride down in the heartbeat <i>response</i> .
Diagnose hard cases; talk to the owner	AI agent (optional, on top)	Reads the daemon's facts + backend state; handles non-deterministic judgment ("GPU throttling", "model won't load", "why am I not earning?") and <i>requests</i> actions via the daemon — never runs its own heartbeat/WG. Scoped key, not the provider key.

Mental model: the **daemon is the body** (senses the machine, keeps it running) · the **backend is the doctor** (independently verifies each body is alive) · the **AI agent is the consultant** (called in for puzzling cases). The body never declares itself healthy — the doctor verifies from outside.

2 Monitoring — three layers, each catching what the one below can't fake

LAYER 1 · SELF-REPORT Daemon heartbeat (~30s) "alive, engine up, tunnel healthy, GPU 64°C, models cached, accepting jobs." Necessary but spoofable — a dead box reports nothing; a faked heartbeat lies.	LAYER 2 · EARNED Backend verification The backend itself probes each provider (real /v1/models + 1-token completion + WG handshake age). Answers "<i>did this box actually serve just now?</i>" — catches "online but 503s."	LAYER 3 · OFF-BOX Dead-man's-switch + alerts External monitor that survives the box <i>and</i> the backend. On online→offline, alerts the provider + the platform; aggregate alarms fire when usable capacity drops.
--	--	--

"Is everything running on **every** machine?" is answered by the **fleet registry + /admin/fleet** (and the agent-readable dcp-fleet CLI): per provider — *daemon alive? engine up? tunnel healthy? verified-serving now?* — all from earned signals.

3 Manage & maintain at fleet scale

- **One daemon, one source, self-updating** — a single platform-served binary every node converges to (no bundled/forked copies); sha256-verified with rollback.
- **One control channel** — the heartbeat *response* is the command bus (`run_mode`, `pull`, `drain`): change fleet behavior without SSHing 500 boxes.
- **Contract is the enforced seam** — heartbeat payload + the daemon's local control API live in `dcp-contracts` and are **CI-checked against the live backend**, so `daemon/backend/desktop/agent` can't silently drift.
- **Structured telemetry, not log-paste** — engine restarts, OOM, WG self-heals, update outcomes as a stream; heartbeats spool offline and backfill on reconnect, so the fleet view has *history*.

4 Status — built vs. gaps

✓ Built today

Daemon GPU+engine introspection, engine watchdog, WG self-heal, local health endpoint, self-update; backend **earned-verification** (`providerVerification.js`); **fleet view + machine-readable dcp-fleet CLI**; **off-box watchdog**.

⚠ Gaps (wiring, not invention)

Alert on offline transition · one daemon source / end the dual-runtime fight · enforce the `run_mode/cap` config the UI already collects · structured telemetry + offline heartbeat backlog · make the contract the enforced seam. (Backlog #1, #6, #7, #9, #11–13.)